ALTO PROTOCOL

AltoConvexCreditLine

Production Security Review

NOT READY FOR PRODUCTION

Read-only assessment of market accounting, Convex custody, rewards, foreclosure, initialization, access control, pause behavior, and operational liveness.

Assessment date: 8 September 2026

Working tree based on commit: 66c0a9a71df3aebe98fda08503de56c9aea2c4fb

Branch: 08-14-CreditLine-init

No Solidity or Markdown source files were modified during this review.

Executive decision

The current implementation should not be deployed to Ethereum production. It has two immediate release blockers: the implementation bytecode exceeds Ethereum's active contract-size limit, and the Credit/Convex variant has no dedicated tests despite unresolved oracle, foreclosure, and reward-policy decisions.

Top priorities before deployment

- Produce deployable bytecode under a locked compiler configuration, then perform full differential and regression validation.
- Finalize and audit the exact frxUSD/DUSD Curve-LP oracle, including depeg, staleness, decimals, and manipulation behavior.
- Add Credit-specific unit, fuzz, invariant, mainnet-fork, and UUPS deployment/upgrade tests.
- Add a collateral-loss reconciliation and foreclosure path that does not depend on a fully healthy Convex withdrawal path.
- Resolve default economics: unpaid fee minting, cumulative bad debt, facility reopening, and pending-reward ownership.

Risk summary

This report contains 22 findings: 4 critical, 4 high, 10 medium, and 4 low. Some findings are conditional deployment or governance risks rather than permissionless exploits. Their conditions are stated explicitly.

Scope and verification

Primary contract reviewed:

```
contracts/lending/credit-market/AltoConvexCreditLine.sol (lines 1-651)
```

Supporting code reviewed included AltoBaseMarket, FixedRateIrm, DUSD, pause and authorization utilities, Credit/Convex interfaces, and the complete Credit Market Markdown documentation. Canonical Convex BaseRewardPool and StashTokenWrapper behavior was compared against the integration assumptions.

Verification performed:

- forge build --sizes: default optimized runtime 26,170 bytes.
- FOUNDRY_PROFILE=deployment forge build --sizes: runtime 26,614 bytes.
- Diagnostic deployment build with via-ir: runtime 22,872 bytes.
- FOUNDRY_PROFILE=market forge test: 523 passed, 0 failed, 0 skipped.
- Repository search: no test references AltoConvexCreditLine or CREDIT_MINT.
- Read-only Ethereum RPC check: documented PID 565 LP, deposit token, gauge, reward pool, Booster, and non-shutdown status matched; the pool had two extra reward contracts at review time.

This is an engineering security review, not a substitute for an independent audit. Static review and unrelated passing tests cannot prove production safety.

Release blockers

CRITICAL F-01

Implementation bytecode cannot currently deploy on Ethereum

The default optimized runtime is 26,170 bytes and the deployment-profile runtime is 26,614 bytes. Ethereum EIP-170 permits at most 24,576 bytes. A UUPS implementation is still a contract deployment and remains subject to the limit. The deployment-profile build exceeds the limit by 2,038 bytes and exits with an error.

Evidence

`README_ALTO_BASE_MARKET_CHANGES.md:252-255`

`https://eips.ethereum.org/EIPS/eip-170`

`https://eips.ethereum.org/EIPS/eip-7907`

Recommended action

Reduce runtime bytecode or adopt a reviewed compiler/architecture change. Via-IR reduced the diagnostic build to 22,872 bytes, but this is not a drop-in production fix: lock compiler settings and run complete differential, invariant, and fork testing before relying on it.

CRITICAL F-02

No Credit/Convex test coverage and material specification remains unresolved

No test under test/ references AltoConvexCreditLine or CREDIT_MINT. The 523 passing lending tests exercise Base, Borrow, Mint, and IRM behavior, not this variant. The repository README explicitly says this implementation is untested, unaudited, and not ready for production. TODO.md still leaves core oracle, LTV, cure-period, foreclosure, reward, and bad-debt policies undecided.

Evidence

`README.md:3-5`

`TODO.md:3-41`

`test/: no AltoConvexCreditLine or CREDIT_MINT references`

Recommended action

Add dedicated unit, fuzz, invariant, Ethereum mainnet-fork, proxy-initialization, and upgrade tests. Resolve every economic and lifecycle TODO before freezing the specification and commissioning an independent audit.

High-severity findings

HIGH F-03

Production LP oracle is unspecified and insufficiently constrained by the market

The market accepts any nonzero price from the configured oracle. It does not enforce freshness, deviation bounds, depeg treatment, decimal normalization, pool-manipulation resistance, or circuit breakers. An inflated price permits excess DUSD minting; a depressed price permits foreclosure; a stale or reverting oracle blocks borrowing and foreclosure. The exact frxUSD/DUSD LP oracle has not been selected in the repository.

Evidence

`AltoConvexCreditLine.sol:392-400`

`AltoBaseMarket.sol:168-180`

`PRICING_LP.md:1-25`

`TODO.md:9-12`

Recommended action

Finalize and independently audit the exact LP oracle. Test underlying-feed failures, DUSD/frxUSD depegs, stale rounds, Curve imbalance/manipulation, virtual-price behavior, decimal combinations, and conservative shutdown behavior.

HIGH F-04

All collateral exits depend on the same Convex withdrawal path

Debt-free withdrawal, governance foreclosure, and emergency exit all call the same rewardPool.withdrawAndUnwrap path. Canonical Convex BaseRewardPool still iterates every dynamic extraRewards entry and calls each external reward contract even when claim=false. One reverting extra-reward contract, excessive external list growth, or a Booster/reward-pool failure can block every collateral exit. Alto's 16-token sweep cap does not bound this Convex loop.

Evidence

`AltoConvexCreditLine.sol:509-532, 631-648`

`README.md:98-101`

`https://github.com/convex-eth/platform/blob/main/contracts/contracts/BaseRewardPool.sol`

Recommended action

Treat Convex reward-manager liveness as a core custody dependency. Add an independent recovery or migration path, monitor every live extra-reward contract, and test failures using an Ethereum fork. Correct README language implying that all extra-reward faults are isolated from withdrawal.

HIGH F-05

No reconciliation path for a Convex collateral shortfall

Position accounting records nominal raw-LP deposits. If actual Convex stake becomes smaller, normal foreclosure attempts to unwrap the larger recorded amount and reverts. Emergency exit can recover only the smaller reported stake, leave position collateral unchanged, and mark Convex inactive. After unpause, borrowing still values the larger recorded collateral, potentially increasing debt against missing LP. Full withdrawal and foreclosure then fail because idle collateral is insufficient.

Evidence

`AltoConvexCreditLine.sol:333-361, 374-405, 509-532, 631-648`

Recommended action

Add explicit loss reconciliation. Before borrowing, enforce an accounted-versus-custodied collateral invariant. Permit emergency foreclosure to seize available collateral while clearing or explicitly writing off debt, and define who absorbs a Convex loss.

HIGH F-06

Unpaid fees and interest remain mintable after foreclosure

Opening fees and accrued interest increase claimableFeesAssets. Foreclosure clears all debt assets and shares but does not reduce claimable fees. The owner or fee recipient may therefore mint amounts whose corresponding debt was written off rather than repaid. Example: after 100 DUSD principal plus 50 DUSD interest is foreclosed, the defaulted 100 remains circulating and another 50 may still be minted as protocol revenue. README calls this intentional while TODO.md says the policy is undecided.

Evidence

`AltoConvexCreditLine.sol:258-299, 333-361`

`README.md:86-89`

`TODO.md:17-19`

Recommended action

Make an explicit economic decision. If only paid fees constitute revenue, separate realized from outstanding fees and write off the unpaid portion at foreclosure. If retained, quantify and cap the additional unbacked-supply exposure.

Medium-severity findings

MEDIUM F-07

Complete DUSD repayment depends on a privileged fee claim

Borrowing mints only net principal, while fees and interest immediately increase debt but remain unminted in claimableFeesAssets. DUSD.burn rejects a burn exceeding this market's per-minter minted counter. Before foreclosure, market debt therefore equals the DUSD minted counter plus unclaimed fees. A full repayment reverts until all claimable fees are minted by the owner or fee recipient. If both are unavailable, the borrower cannot clear shares or release collateral; a tight token ceiling can require carefully staged claims and repayments.

Evidence

`AltoConvexCreditLine.sol:243-299, 364-370`

`AltoBaseMarket.sol:437-501, 525-547`

`DUSD.sol:33-53`

Recommended action

Make fee realization permissionless to the fixed recipient, realize fees automatically during repayment, or formally specify and test a keeper/minter-ceiling invariant that guarantees final repayment liveness.

MEDIUM F-08

Foreclosure reopens accounting capacity while bad-debt DUSD remains outstanding

Foreclosure resets totalBorrowed to zero and leaves the credit limit unchanged, but burns no circulating DUSD. Market accounting permits another complete draw. DUSD's per-minter counter still includes the defaulted issuance: a tight token ceiling blocks later draws, while raising it permits cumulative defaulted supply to exceed the market's advertised credit limit across foreclosure cycles.

Evidence

`AltoConvexCreditLine.sol:113-135, 333-361`

`DUSD.sol:17-20, 33-53`

`README.md:80-87`

`TODO.md:29-38`

Recommended action

Close or reduce the facility after foreclosure until explicit recapitalization, or document that creditLimit covers only current booked debt and enforce a separate cumulative bad-debt/minter-exposure cap.

MEDIUM F-09

Successful foreclosure charges interest for the paused interval

Pausing accrues through the pause timestamp. Normal unpause resets FixedRateIrm.lastUpdate, excluding the paused interval. governanceLiquidate remains callable while paused and unconditionally accrues, charging the time since pause. A position can become foreclosable solely from paused-time interest. A failed foreclosure rolls this accrual back; a successful one commits it and also increases claimable fees.

Evidence

`AltoConvexCreditLine.sol:333-345`

`AltoBaseMarket.sol:747-754`

`FixedRateIrm.sol:59-65, 95-119`

Recommended action

Skip foreclosure accrual while fully paused, disallow foreclosure while paused, or make the IRM explicitly pause-aware. Document whichever economic behavior is selected.

MEDIUM F-10

Entering emergency pause depends on a healthy IRM and non-overflowing accounting

Pausable.pause invokes the Base _onPause hook before setting the pause flag. That hook calls the Credit interest accrual path, including an external IRM call and checked uint128 additions. A reverting IRM or overflow prevents PAUSE_TYPE from being entered. emergencyExitConvex requires PAUSE_TYPE, so the supposed escape hatch is unavailable under the same failure. FROZEN_TYPE can still stop new draws, and the owner can normally install a fresh IRM with skipAccrueInterest, but this depends on owner response and fresh infrastructure.

Evidence

`Pausable.sol:26-32`

`AltoBaseMarket.sol:747-754`

`AltoConvexCreditLine.sol:284-292, 509-515`

Recommended action

Permit emergency exit under a reliable frozen state or add a narrowly scoped no-accrual emergency-pause/exit path. Preserve an explicit debt snapshot policy.

MEDIUM F-11

Malformed reward tokens can disable claims and their own removal

Every configured token is synchronously queried and transferred in one atomic claim. A token whose balanceOf or transfer reverts disables all reward forwarding. removeRewardToken also invokes balanceOf and requires a zero balance, so the bad entry may be impossible to remove without upgrading. Contract code existence is not proof of ERC-20 compliance or future behavior.

Evidence

`AltoConvexCreditLine.sol:436-455, 476-505, 586-600`

`README.md:125-140`

Recommended action

Add a force-quarantine/removal path that does not call the token, plus a separately constrained recovery/sweep mechanism. Consider per-token isolation so one reward cannot block all others.

MEDIUM F-12

Initialization does not validate the actual tokens paid by Convex

Only the primary reward token must be allowlisted. PID 565 currently pays CRV and causes underlying CVX and WFRAX distributions through extra-reward wrappers. Convex StashTokenWrapper.transfer sends the underlying token, while balanceOf(non-reward-pool) reports zero. If deployment config contains wrapper addresses rather than their underlying token addresses, claimRewards observes zero wrappers and leaves the actual rewards in the credit line.

Evidence

`AltoConvexCreditLine.sol:535-600`

`CONVEX_NOTES.md:126-169`

`https://github.com/convex-eth/platform/blob/main/contracts/contracts/StashTokenWrapper.sol`

Recommended action

For PID 565, fork-test the actual CRV, CVX, and WFRAX payout flow. Document how wrapper.token() is resolved, validate the initial list, and continuously monitor extra-reward changes.

MEDIUM F-13

Pending-reward ownership after foreclosure is undefined

Foreclosure withdraws with claim=false. Pending rewards remain claimable later, and rewardRecipient remains controlled by the former borrower even after governance writes off debt and seizes LP. The TODO explicitly leaves ownership of these rewards undecided.

Evidence

`AltoConvexCreditLine.sol:333-361, 436-467, 631-648`

`TODO.md:21-24`

Recommended action

Specify whether pending pre-foreclosure rewards belong to the borrower, governance, or the loss reserve. If governance should receive them, design settlement without importing malformed-reward-token DoS into the foreclosure transaction.

MEDIUM F-14

Governance can change live foreclosure terms immediately

The owner may replace the oracle and alter maxLtv or foreclosureLtv while debt exists. The only enforced relationship is maxLtv < foreclosureLtv, so the buffer may be one unit of 1e18 precision. There is no delayed activation, grace period, minimum buffer, or live-debt restriction. Governance can change valuation or lower the threshold and foreclose in the same execution.

Evidence

`AltoConvexCreditLine.sol:138-166, 384-400`

`AltoBaseMarket.sol:168-195`

`TODO.md:3-12`

Recommended action

Enforce a minimum buffer and delayed activation, freeze sensitive changes while debt exists, or place all powers behind a clearly documented timelock and borrower-notice policy.

MEDIUM F-15

Immutable credit account and dependencies create permanent-lock scenarios

creditAccount cannot be rotated. If its EOA key is lost or its smart-account logic becomes unable to call the market, healthy debt-free LP cannot be withdrawn unless a delegate was already authorized; governance foreclosure requires an actual threshold breach. Reward-recipient rotation also becomes unavailable. Booster, pool, and token dependencies likewise have no normal migration path, making UUPS ownership the only generic recovery mechanism.

Evidence

`AltoConvexCreditLine.sol:37, 111, 200-225, 408-413, 458-467`

Recommended action

Use a resilient smart account, add delayed/two-step rotation restricted to safe states, define a paused recovery policy, verify token roles continuously, and never renounce upgrade ownership while these dependencies remain immutable.

MEDIUM F-16

Direct inheritance weakened the generic IRM reentrancy boundary

The predecessor Credit implementation exposed accrueInterest with nonReentrant. The inherited Base entry point is unguarded and dispatches into Credit _accrueInterest, which calls the configured IRM. IRM_TYPE == FIXED_RATE proves only a returned enum value, not code identity, immutability, or callback behavior. A malicious or upgradeable IRM can recurse, exhaust gas, or manipulate repeated accrual. The repository FixedRateIrm does not callback and is safe under that exact trust assumption.

Evidence

`AltoBaseMarket.sol:550-553`

`AltoConvexCreditLine.sol:168-179, 283-292`

Recommended action

Allowlist an immutable audited IRM implementation or factory. If arbitrary fixed-rate implementations remain supported, restore a reentrancy boundary and test adversarial callbacks.

Conditional deployment hazards

CRITICAL F-17

An uninitialized proxy is fully takeable

The implementation constructor correctly disables its own initializer, but a separately deployed proxy with empty initialization data can be initialized by anyone. The attacker can choose owner, credit account, tokens, IRM, oracle, and Convex configuration, then control upgrades. No Credit deployment script currently exists in the repository.

Evidence

`AltoConvexCreditLine.sol:72-82`

`https://docs.openzeppelin.com/upgrades-plugins`

Recommended action

Create and initialize the UUPS proxy atomically with encoded initialize calldata. Never use a two-transaction proxy-deploy then initialize sequence.

CRITICAL F-18

Current implementation is storage-incompatible with any prior Credit proxy

Direct AltoBaseMarket inheritance introduced Base fields and a different storage placement relative to the deleted AltoCreditMarketBase. Installing this implementation over any proxy initialized with the prior layout corrupts state. The current documentation assumes the Credit line is undeployed.

Evidence

`README_ALTO_BASE_MARKET_CHANGES.md:194-215`

Recommended action

Deploy a fresh proxy. If any prior proxy or test deployment holds meaningful state, perform explicit slot-level migration analysis instead of upgrading it directly.

Low-severity and operational findings

LOW F-19

FixedRateIrm market binding is permissionless first-writer-wins

FixedRateIrm.setMarket allows any caller to bind a fresh IRM once. An observer can bind the IRM before Credit initialization or front-run setIrm, making that IRM instance unusable by the intended market. This is a deployment/configuration DoS rather than direct fund theft.

Evidence

`FixedRateIrm.sol:47-56`

`AltoBaseMarket.sol:133-135`

Recommended action

Deploy and bind a fresh IRM atomically through a batch/factory, use private execution, or restrict/precommit the allowed market in FixedRateIrm.

LOW F-20

Virtual-share rounding can leave orphan global debt assets

After sufficiently large interest growth, repaying all real borrow shares can round the asset repayment below totalBorrowed.assets. Position and global shares become zero while global assets remain positive. Collateral release checks only position borrow shares, so collateral can leave while the orphan amount consumes future capacity and may continue accruing interest.

Evidence

`AltoBaseMarket.sol:525-547`

`AssetShareConversionMath.sol:45-57`

`AltoConvexCreditLine.sol:209-225, 373-405`

Recommended action

When the sole account repays all global shares, explicitly reconcile remaining global asset dust, or require both global assets and shares to be zero before final collateral release.

LOW F-21

Reward removal is permissionlessly dust-griefable

Removal requires the market token balance to be exactly zero. Anyone can transfer one smallest unit to the market, or invoke the public Convex reward claim for the market, and front-run a standalone removal repeatedly. Removed or unconfigured future rewards can also remain stranded until the token is re-added or the implementation upgraded.

Evidence

`AltoConvexCreditLine.sol:436-455, 476-505`

`README.md:134-140`

Recommended action

Allow removal with explicit forwarding/quarantine semantics, or guarantee an atomic claim-and-remove batch through the governance owner. Add a constrained surplus recovery path that cannot sweep accounted collateral.

LOW F-22

Pauser and ownership lifecycle contain operational footguns

Inherited setPauser accepts zero addresses and arbitrary pause-type values, unlike the predecessor Credit implementation. Ownership transfer does not revoke the former owner's existing PAUSE_TYPE and FROZEN_TYPE permissions or automatically grant them to the new owner. Renouncing ownership removes upgrades, foreclosure, emergency exit, and recovery setters while old pauser permissions may remain.

Evidence

`AltoBaseMarket.sol:251-258`

`Pausable.sol:10-39`

`AltoOwnable2StepUpgradeable.sol:43-80`

Recommended action

Use a deployment and ownership-transfer runbook that explicitly grants and revokes both pause roles. Prefer a resilient timelock/multisig owner and operationally prohibit renounceOwnership and zero-address skip transfers.

Required production acceptance checklist

- Implementation runtime is below the active chain limit using committed, reproducible compiler settings.
- Every Credit/Convex finding above is fixed or explicitly accepted in a signed threat model.
- The exact oracle implementation and parameters are audited and deployed.
- Credit unit, fuzz, invariant, fork, proxy, pause, foreclosure, and loss-path tests pass.

- DUSD minter and burner roles, ceiling headroom, claim ordering, and bad-debt accounting are tested end-to-end.
- PID 565 actual reward payouts are verified as CRV/CVX/WFRAX rather than wrapper balances.
- Convex extra-reward contracts, shutdown status, and withdrawal behavior are continuously monitored.
- Emergency governance can act within the required time and does not depend on a single compromised signer.
- Proxy deployment initializes atomically; no prior incompatible Credit proxy is upgraded.
- An independent audit is completed against the exact bytecode intended for deployment.

Review conclusion

The core design can be made production-oriented, but its present form combines an undeployable implementation with unresolved oracle/default economics and a non-independent Convex escape path. The highest-risk engineering work is not cosmetic: bytecode delivery, collateral-loss reconciliation, oracle security, repayment liveness, and full Credit-specific testing must be completed before production approval.